

INTERNATIONAL CONFERENCE ON RECENT TRENDS IN ADVANCED COMPUTING
2019, ICRTAC 2019

A Genetic Algorithm for the Snake-in-the-Box Problem using Modified Edge Recombination

Rashmi K A^{a,*}, Kalpana B^b

^a*Intuit, Ecospace Building 8, Bellandur, Bangalore, Karnataka, India - 560103*

^b*PSG College of Technology, Coimbatore, Tamil Nadu, India - 641004*

Abstract

Snake-in-the-box is a graph theory problem of finding the longest path in a hypercube with some constraints. As the dimension of the hypercube increases, the number of paths grow, increasing the search space exponentially. In the past, genetic algorithms have been known to work well with problems having such large search spaces. In this paper, a genetic algorithm using a modification of the edge recombination operator is presented to find the longest snake in dimensions one to nine with lesser edge failures.

© 2019 The Authors. Published by Elsevier B.V.

This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>)

Peer-review under responsibility of the scientific committee of the INTERNATIONAL CONFERENCE ON RECENT TRENDS IN ADVANCED COMPUTING 2019.

Keywords: genetic algorithms; snake-in-the-box; edge-NN

1. Introduction

In 1958, W.H. Kautz coined the term ‘snake-in-the-box’ for a set of error checking codes in which each of the code elements were spaced at a Hamming distance of one from each other. These codes can be useful in error detection for digitalized analog data[1] so that analog-to-digital conversion can be done with minimal errors. Kautz

* Corresponding author. Tel.: +91 9443314141.

E-mail address: k.a.rashmi04@gmail.com

defined these codes with the unit distance property as a chain of vertices in which no two vertices are adjacent, except the ones appearing consecutively. The snake-in-the-box problem is finding the longest chain of such vertices, so that the maximal number of elements can be incorporated into codes. While the initial application of this problem was for error checking codes, several new applications such as identification of tracks in servo fields of a disk drive[2] and DNF simplification have also been identified.

The snake-in-the-box can be considered as a graph theory problem which involves finding the longest path in a hypercube of dimension n (Q_n) with certain constraints. The n -dimensional hypercube graph Q_n has 2^n nodes and each node has n neighbors, consisting of all the nodes which differ by only one bit. In a 3-dimensional hypercube, there are 8 nodes and each node has 3 neighbors. The nodes are generally denoted as words of length n . Therefore, in Q_3 , the nodes are 001,010,011,100,101,110,111. In this paper, the nodes will be referred to using their decimal representation (011 as node 3). A snake, as defined above, is a cycle in a hypercube without any chords in between the nodes[3]. Figure 1a. and 1b. show two possible snakes within the 3-dimensional hypercube.



Fig. 1. (a) A snake of sequence [6,4,0,1,3]; (b) A snake of sequence [1,5,7,6,2].

As the dimension increases, the number of possible paths rises exponentially, creating a combinatorial explosion and making this an NP-Complete problem[4].

Several approaches to solve this problem have been proposed. These include evolutionary algorithms, heuristic search methods and other mathematical approaches. In this paper, a simple genetic algorithm (GA) is proposed that succeeds in discovering the longest possible snakes in a dimension. Genetic algorithms are suboptimal search methods based on the theory of survival of the fittest[5]. They have shown promising results in the past for this problem, generating new world records for snake lengths in higher dimensions ($n \geq 7$). In this work, the GA has been applied for dimensions 1 through 8 and has successfully generated the longest snakes as found so far through other approaches.

2. Proposed methodology

A genetic algorithm involves two main steps – generating an initial population and modifying the population by reproduction. A population consists of a set of individuals, each of which is associated with a fitness. Potter et al. defined two approaches for denoting individuals for this problem – node sequence and transition sequence[6]. In this work, an individual is denoted as a sequence of nodes $[v_1, v_2, \dots, v_n]$, such that for every two nodes (v_i, v_j) which are adjacent to each other, v_i and v_j differ by 1 bit and there is an edge between them. This is the node sequence approach. A random generator function can be used to generate the initial individuals in the population. The algorithm picks neighbors randomly, with the constraint of not picking the same node again. For dimensions greater than 6, the length of the individual is based on the expression proposed by Snevily H. S. [7] and is given by:

$$U \leq 2^{d-1} - \frac{2^{d-1}}{20d-41}, d \leq 7 \quad (1)$$

where U is the upper bound of the longest possible length for a snake in a given dimension d . For $d = 7$, the value of U is calculated to be 63.3. The individual length is then considered to be 64 in the experiment. Similarly, for dimensions 7 to 9, the length of the individual is listed in Table 1.

Table 1. Individual length.

Dimension (n)	Upper bound(U)	Individual length
7	63.3	63
8	126.9	127
9	254.15	254

For every individual, the fitness is calculated using a fitness function. The fitness function is generally an objective function and is a measure to decide which of the individuals make it to the next generation. New generations can be formed from the individuals in the initial population, keeping the fitness value as a judge for mate selection and reproduction. The GENITOR algorithm is used for reproduction. GENITOR has a one-at-time reproduction scheme[8] and has shown promising results in the past amongst other algorithms[9]. In GENITOR, the weakest out of all the individuals in the population is replaced by a crossover of the stronger individuals. For mate selection, GENITOR uses the rank-based selection method. This is further proved to be a good option in the experiment conducted by Razali et al. in which several algorithms such as tournament selection, proportional roulette wheel selection and rank based selection have been tried and tested[10]. Out of these, the rank-based selection showed better results even though the convergence rate was slower than in the case of the other two selection mechanisms. In rank-based roulette wheel selection, individuals are selected based on their ranks which are calculated after sorting them according to their fitness. This mechanism ensures that the probability of an individual being selected is based on its fitness rank relative to the entire population[11]. After the two parents are chosen, the offspring is generated using a modified version of the edge recombination algorithm.

The edge recombination operator(ERO) is a genetic operator used for crossover that is applied in problems where the nodes have adjacency information between them that need to be preserved. In ERO, an adjacency table of all the nodes in both the parents is formed. Starting from a node, the next node is picked by choosing the neighbor which has the least amount of nodes in its adjacency list. This is done continuously to build the child from two parents. In the past, a modification of the ERO called the enhanced edge recombination operator[9] has been used to develop genetic algorithms for the snake-in-the-box problem. This operator preserves common subsequences in both the parents. When the adjacency list is built for a node, if a neighbor is adjacent in both parents, the sign is inverted for it. Giving priority to the negative nodes, neighbors are then picked. If there are no negative neighbors, the nearest neighbor is picked as in ERO. However, the previous approaches that have used this operator have given bad results and as an alternative, n-point crossover with a transition sequence approach is generally applied. In this paper, we use a modification of the edge recombination operator called Edge-NN(Edge-2, nearest neighbor)[12]. This overcomes a major disadvantage of ERO which is edge failure[13]. An edge failure occurs when there are no more nodes in the adjacency list of the current node. This is typically solved by picking an element randomly and then proceeding on with the algorithm. The edge-NN algorithm reduces the number of edge failures and has shown a faster convergence rate than the regular operator as well as the enhanced edge operator.

Parent 1 - [12, 14, 6, 7, 3, 11, 9, 1]

Parent 2 - [1, 5, 7, 6, 14, 10, 11, 3]

↓
Child - [14, 6, 7]

Table 2. Edge-NN Example

Node	Adjacency list	Comments
1	5	-
3	-11	11 appears as 3's neighbour in both parents, 7 is visited already
5	1	7 is visited already
7	3,5,6	6 is visited already
9	1,11	-

10	11,14	14 is visited already
11	3,9,10	-
12	14	-

In Edge-NN, a part of parent1 is inherited directly to the offspring. The adjacency table is built for the remaining nodes in parent1 and all the nodes in parent2. Considering the last node of the inherited part in the offspring to be the current node, all its remaining occurrences are removed. The neighbors are then picked in the same way as in the enhanced edge recombination algorithm, giving priority to negative nodes. When a node has no neighbors, instead of choosing randomly, the node nearest to the current node is picked, with ties broken randomly. An example for the Edge-NN algorithm is given in Figure 2. After the adjacency table is constructed, the next nodes are picked to build the offspring.

Current node = 7, next node = 3	Child = [14,6,7,3]
Current node = 3, next node = 11	Child = [14,6,7,3,11]
Current node = 11, next node = 10	Child = [14,6,7,3,11,10]
Current node = 10, edge failure(both 14 and 11 are already visited)	
Unvisited nodes = {1,5,9,12}	
Nearest neighbor = 9	Child = [14,6,7,3,11,10,9]
Current node = 9, next node = 1	Child = [14,6,7,3,11,10,9,1]

3. Experiment

In the experiment, individuals are initially created using the random generation function. For the sake of performance, neighbors were computed prior to the trails. Although seeding of trials with the best results of the previous trials have shown good results in the past[6], no seeding has been done in this work. For dimensions greater than 5, the population sizes ranged between 6,000 to 30,000. The fitness of the individuals in the population is then calculated by finding the length of the longest sub-snake in the individual[6]. The number of generations created were between the ranges 10,000 to 40,000. The number of edge failures that occurred were found to be lesser than in regular ERO. The trial result is given below in Table 3.

Table 3. Edge failures for ERO and edge-NN.

Dimension (n)	Average ERO edge failures	Average Edge-NN edge failures
4	22	19
5	57	51
6	112	96

The proposed genetic algorithm is used to find snakes in the hypercubes Q_1 to Q_9 . This algorithm aims to find the longest snakes using the node sequence approach. Reducing the number of edge failures is crucial in the node sequence approach since every time there is a node failure, the next nodes are selected by random. Random selection prohibits the assembling of long snakes by breaking the chain of neighboring nodes. The usage of the edge-NN operator has shown reduction in the number of edge failures. This indicates that the edge-NN operator can be used to find snakes of higher dimensions. The snake lengths found by the algorithm for the dimensions 1 through 9 are given in Table 4.

Table 4. Experiment result

Dimension (n)	Snake length
1	1
2	2
3	4
4	7
5	13
6	26
7	50
8	98
9	179

4. Conclusion

The results of using edge-NN to solve this problem are very encouraging. The drawbacks of the edge recombination operator have led most approaches to solve this problem to be in transition sequence approach rather than node sequence[14]. Modification of this operator should prove useful to solve the problem in the node sequence approach in even higher dimensions. Further improvements can be done by adding heuristics and other optimizations to this problem to achieve even faster convergence. This can also be extended to searching for coils(closed snakes) in hypercubes.

References

- [1] Kautz, William H.. (1958) "Unit-distance error-checking codes." *Electronic Computers, IRE Transactions on EC* **7**(2):179–180.
- [2] Blaum, Mario and Etzion, Tzvi, inventors; INTOVOICE Corporation, assignee. Use of snake-in-the-box codes for reliable identification of tracks in servo fields of a disk drive. US Patent 6,496,312. 2001 Jan 05.
- [3] Abbott, H.L. and Meir Katchalski. (1987) "On the snake in the box problem", *J. Comb. Theory, Ser. B* **45**: 13-24.
- [4] Garey, M. R., and D. Randolph Johnson. "Computers and Intractability A Guide to the Theory of NP-Completeness." New York: NY, USA; 1978.
- [5] Goldberg, D.E.. "Genetic Algorithms in Search, Optimization, and Machine Learning." Boston: MA, USA; 1989.
- [6] Potter, Walter D., R.W. Robinson, J.A. Miller, K.J. Kochut, D.Z. Redys. "Using The Genetic Algorithm to Find Snake-In-The-Box-Codes." In: Industrial and Engineering Applications of Artificial Intelligence and Expert Systems - Proceedings of the Seventh International Conference: 1994; Austin, Texas.
- [7] Snevily, Hunter S.. (1994) "The Snake-in-the-Box Problem: A New Upper Bound." *Discrete Mathematics*. **133**(1-3):307-314.
- [8] Whitley, L. Darrell. "The GENITOR algorithm and Selection Pressure: Why rankbased allocation of reproductive trials is best." In: Proceedings of 3rd International Conference on Genetic Algorithms: 1989, pp. 116–121.
- [9] Starkweather, T., S. McDaniel, K. Mathias, D. Whitley, and C. Whitley, "A Comparison of Genetic Sequencing Operators," in Proceedings of the Fourth International Conference on Genetic Algorithms: 1991, pp. 69-76
- [10] Noraini Mohd Razali, John Geraghty. "Genetic Algorithm Performance with Different Selection Strategies in Solving TSP", In: Proceedings of the World Congress on Engineering Volume-02: 2011 July 6 - 8.
- [11] Katayama, Kengo et al. "Performance analysis for crossover operators of genetic algorithm." *Systems and Computers in Japan* **30** (1999): 20-30.
- [12] Yiu-Cheung Tang A., Leung KS. "A modified edge recombination operator for the Travelling Salesman Problem". In: Davidor Y., Schwefel HP., Männer R. (eds) Parallel Problem Solving from Nature — PPSN III. PPSN 1994. Lecture Notes in Computer Science, vol 866; Berlin, Heidelberg: Springer, 1994.
- [13] Mathias, K., and Whitley, D. (1992) "Genetic operators, the fitness landscape and the traveling salesman problem." *Parallel Problem Solving from Nature — PPSN 2*.
- [14] Bitterman, Derrick Scott. "New Lower Bounds for the Snake-in-the-Box Problem [dissertation]". The University of Georgia; 2004.